

Arachne: Orchestrating Cascades for Efficient Text-to-Video Model Training

The rising demand for AI-generated videos is fueled by advances in large-scale Text-to-Video (T2V) models, trained on extensive datasets of video clips spanning diverse resolutions and durations. To address this data heterogeneity, training commonly employs a bucketing strategy that groups samples into discrete buckets to improve efficiency. However, we find bucketing struggles to scale efficiently with growing compute and data volumes under static parallelism techniques, such as data and sequence parallelism, resulting in considerable workload imbalance and resource under-utilization.

In this paper, we present Arachne, a novel training framework for efficient large-scale T2V model training. The core idea behind Arachne is to break down the training into fine-grained computational units, called *cascades*, and to orchestrate their execution through coordinated spatial and temporal optimization to maximize training efficiency. At each training step, Arachne first performs temporal optimization by generating an optimized execution plan that defines the scheduling and parallelism configuration for each cascade. It then applies spatial optimization by performing a topology-aware mapping of these cascades onto specific devices to minimize communication overhead. Finally, Arachne dispatches the cascades to carry out the plan, ensuring execution correctness under this flexible scheme. Our comprehensive evaluation demonstrates that Arachne not only delivers significant speedups over leading training frameworks, reducing iteration time by up to 65%, but also exhibits a positive scaling trend where its performance advantage amplifies as the training scale grows.

1 Introduction

Video synthesis with deep generative models is transforming diverse domains including media content creation [1, 3, 45, 49], interactive video games [6, 51, 57], and world simulation [11, 15, 39]. Among these, Text-to-Video (T2V) models emerge as the leading and rapidly advancing frontier, driven by advances in generative algorithms [16, 30, 31, 43], scalable model architectures [14, 34, 38], and unprecedented availability of compute and data resources [20, 36, 54].

Current state-of-the-art T2V models [5, 23, 49, 52, 56], have converged on the latent diffusion paradigm [43] as their architectural blueprint. As illustrated in Fig. 1, this paradigm involves a Variational Autoencoder (VAE) [21] first compressing video into a latent space, where a Diffusion Transformer (DiT) [38], guided by a text encoder, is then trained to predict artificially added noise. Here, the success of this paradigm in generating high-quality video relies heavily on training with massive datasets [23, 49, 60].

Training on these massive datasets is typically accelerated using data parallelism (DP) [28], where the workload is evenly distributed across multiple devices. However, due to the data's inherent heterogeneity, a local batch randomly sampled for each DP rank will contain a mix of videos with varying durations and resolutions, making it difficult to efficiently organize the data into uniform batches for training. The widely adopted solution is the *bucketing strategy* [23, 46, 60], which pre-groups videos by similar resolutions and durations so that each DP rank can sample a local batch from a single bucket. In addition, for buckets containing long sequences that exceed single-device memory, sequence parallelism (SP) [18, 24, 26, 29] is utilized to partition a single data sequence across a dedicated group of devices.

Challenges. However, while bucketing ensures uniformity within each local batch, it still leaves a considerable workload imbalance across DP ranks. The strategy attempts to mitigate this issue

by incorporating adaptive batch sizing¹, but such adjustments are fundamentally constrained by GPU memory limits and the rigidity of static parallelism. This is clearly illustrated by the example in Fig. 2. In a representative training step in the T2V model HunyuanVideo [23], executed on four DP=4 ranks, each with an SP=4 degree, four distinct batches from separate buckets are fed into the SP groups, with sequence lengths varying from 13 to 113 frames. Due to the static parallelism configuration applied to all groups, the rank processing the longest sequence (113 frames) takes 58.6 seconds, whereas the one with the shortest (13 frames) completes in only 6.0 seconds, resulting in 52.6 seconds of idle time for the fastest rank. This example shows the substantial resource underutilization inherent in current training schemes.

The challenge of mitigating inefficiencies from static parallelism under data heterogeneity has spurred the development of *dynamic parallelism approaches* in the Large Language Model (LLM) field, primarily targeting the communication overhead of short text sequences [12, 55]. However, such a focus is mismatched with the realities of T2V generation, where the combination of significantly longer video sequences and compute-intensive DiT architectures makes computation, not communication, the defining performance bottleneck (more details can be seen in §3).

Proposed Method. This paper proposes Arachne, a novel training framework tailored for T2V model training. At its core, Arachne introduces a unified strategy that integrates fine-grained workload decomposition with dynamic spatio-temporal orchestration. It first breaks down the training process into minimal schedulable units known as *cascades*, each a self-contained task defined by a bespoke configuration that specifies its target model components, input data, parallelization strategy, and scheduling parameters. This decomposition empowers Arachne to execute these cascades with flexibility and efficiency via spatio-temporal orchestration.

Specifically, Arachne introduces three components that work collaboratively to optimize T2V training. ① *Cascade-level parallelism planner* performs temporal optimization by decomposing the training iteration into a Directed Acyclic Graph (DAG) of dependent cascades, co-optimizing the launch time and parallel configuration for each to minimize the theoretical makespan on abstract, homogeneous hardware. ② *Topology-aware resource mapper* then performs spatial optimization by taking the planner’s logical, topology-agnostic plan and, considering the physical network topology, mapping each cascade onto a specific set of GPUs to create a communication-efficient physical execution plan. Finally, ③ *Runtime executor* enacts the concrete spatio-temporal plan by coordinating data handoff across dependent cascades and regulating gradient accumulation within parallelism groups, thereby ensuring both correctness and efficiency.

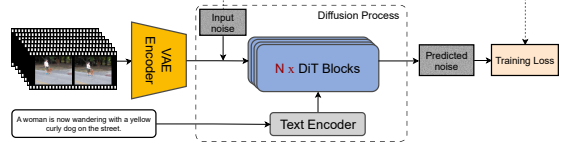


Fig. 1. Training workflow of a typical T2V model, consisting of a VAE, a **text encoder**, and multiple **DiT blocks**.

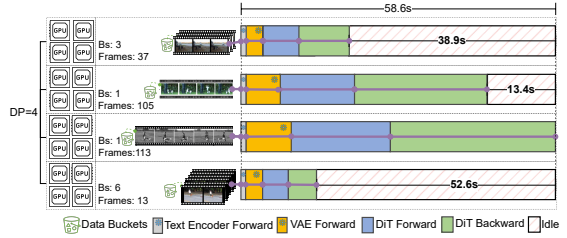


Fig. 2. Illustration of a T2V model training step under the default bucketing (4 frame-length buckets) and parallelism configuration (DP=4, SP=4), showing **significant GPU idle time (hatched regions)** induced by the gradient synchronization barrier following the DiT backward pass.

¹Buckets with shorter sequences are assigned larger batch sizes to equalize the total tokens (i.e., $\text{batch_size} \times \text{sequence_length}$) processed per rank, subject to GPU memory constraints

We evaluate Arachne’s performance on three large-scale T2V models, including Wan2.1 [52], CogVideoX [56], and HunyuanVideo [23], and compare it with two leading training frameworks—Megatron-LM [44] and DeepSpeed [40]—as well as FlexSP [55], a representative training framework tailored for variable-length corpora. Our experimental results show that, under realistic T2V training scenarios, Arachne achieves significant end-to-end speedups, reducing average iteration time by up to 65%, 59%, and 35% compared to these frameworks, respectively. To further evaluate scalability, we examine Arachne across diverse model architectures, workload heterogeneities, and cluster sizes, which reveals a clear **scaling trend**: as training scale increases, the speedup achieved by Arachne grows correspondingly, reflecting the “scaling-law” behavior observed in model performance under realistic large-scale training scenarios.

Contributions. We make the following contributions:

- (1) We present Arachne, a novel training framework designed for efficient large-scale T2V model training. Its design is informed by a systematic analysis of T2V’s unique workload characteristics, revealing performance bottlenecks distinct from those encountered in the LLM domain.
- (2) We propose a cascade-level spatio-temporal orchestration paradigm that mitigates workload imbalance arising from data heterogeneity. This paradigm temporally optimizes the launch time and parallel configuration for each fine-grained cascade, while spatially mapping it to communication-efficient GPU placements.
- (3) Through comprehensive experiments, we show that Arachne achieves significant and consistent speedups over leading baselines, demonstrating both effectiveness and robust scalability across diverse T2V models, workload heterogeneities, and cluster configurations.
- (4) We will open-source the full implementation of Arachne and the T2V training workloads used in our evaluation, fostering reproducibility and enabling further research.

2 Background

This section provides an overview of the core architectures, training paradigms, and parallelization strategies used in modern T2V models.

2.1 T2V Diffusion Models

Modern T2V models predominantly adopt the latent diffusion paradigm [43], as illustrated in Fig. 1, comprising three key components: a VAE for compression, a text encoder for conditioning, and several DiT blocks for denoising.

Spatio-Temporal Compression via VAE. The VAE’s role is to perform spatio-temporal compression by encoding video clips from pixel space into a compact latent representation. The architecture of these VAEs has progressively evolved to better capture temporal dynamics. Early 2D VAEs [3] processed frames independently, often causing temporal inconsistencies. This led to intermediate 2.5D VAEs [49] with interleaved spatial and temporal convolutions, and ultimately to the current state-of-the-art 3D Causal VAEs [58], which use full spatio-temporal convolutions to produce high-quality, temporally coherent latents.

Text-Conditioned Denoising in Latent Space. The core generative process occurs in this latent space. After noise is added to a video’s latent representation, a DiT, the de facto backbone for this task, is trained to reverse the noising process under the guidance of a text encoder. A key architectural differentiator among state-of-the-art models is the mechanism used to fuse the text conditions with the video tokens. Approaches typically fall into two categories: combined self-attention [23, 56], where text and video tokens are concatenated into a single sequence, or cross-attention injection [52], where text tokens provide semantic guidance to video tokens serving

as keys and values. The DiT’s output is then used to compute a training loss based on a chosen paradigm, such as DDPM [16] or Flow Matching [31, 33].

2.2 T2V Training Strategies

Effectively training of T2V models on large-scale datasets relies on two key strategies. The first is *multi-stage curriculum learning*, which ensures stable convergence by progressing from simple to complex tasks. The second is the *bucketing strategy*, which improves batching efficiency for heterogeneous data.

Curriculum Strategy. Curriculum learning is a standard technique that makes training high-resolution models computationally tractable [7, 23, 60]. This coarse-to-fine approach typically begins with pre-training on massive, low-resolution datasets (e.g., 240p-360p) to learn foundational concepts like text-video alignment. This is followed by progressive fine-tuning on smaller, high-quality datasets at gradually increasing resolutions (e.g., 480p-1080p). This staged regimen not only improves final visual quality but also enhances training stability.

Bucketing Strategy. As a practical compromise between naive padding and more complex alternatives like sequence packing [25] (which is difficult to apply efficiently to multi-dimensional video), the bucketing strategy [23, 46, 60] efficiently organizes heterogeneous video data for training. The technique simply involves pre-categorizing videos into discrete “buckets”, each defined by specific dimensions such as resolution and duration.

2.3 Parallelisms in T2V Training

While model parallelism techniques like tensor (TP) [44] and pipeline parallelism (PP) [13, 17] are essential for training LLM with massive parameter counts, they are less central to T2V training at the current stage: T2V models are constrained by substantial activation memory required to process exceptionally long video sequences, rather than by parameter size. Consequently, the dominant strategies for T2V are a family of *data-centric approaches* that focus on scaling throughput and managing long-context memory: data parallelism, sequence parallelism, and context parallelism.

Data Parallelism (DP). DP replicates the model on each device while sharding the training data. During each step, all devices process their local data in parallel before a global gradient synchronization ensures a consistent model update. This approach is often enhanced with optimizers like ZeRO [40] that also partition optimizer states, gradients, and parameters to further reduce per-device memory.

Sequence Parallelism (SP). SP refers to a family of techniques designed to tackle the activation memory bottleneck that arises from long sequences. Implementations of SP differ significantly in their approach. Ulysses-style SP [18] is a head-wise approach that uses a pair of All-to-All communications to transform the tensor layout from sequence-partitioned $[B, H, S/N, D]$ to head-partitioned $[B, H/N, S, D]$ to compute self-attention on the full sequence for a subset of heads, before a second All-to-All reverts the output to the original layout. In comparison, Megatron-style SP [24] partitions only specific operations like dropout and layer norm, often as a memory-saving complement to TP.

Context Parallelism (CP). Unlike the head-wise partitioning of Ulysses-style SP, CP partitions the Q , K , and V tensors along the sequence dimension across multiple devices [4, 27, 32, 37]. The attention computation is then performed block-wise: each device holds its local Q slice and iteratively processes incoming Key (K) and Value (V) slices as they are circulated from other devices in a ring pattern. The key advantage of this approach is that the ring-based communication can be effectively overlapped with the block-wise computation, significantly hiding network latency.

3 Observations

Existing distributed training systems, heavily optimized for LLMs, fall short for T2V generation, whose distinct sequence lengths and computation patterns create new bottlenecks. We highlight two key observations that expose this mismatch and motivate a T2V-aware system design.

Observation 1: Right-shifted, concentrated, and discrete distribution of video workloads. The primary distinction between T2V and LLM workloads lies in the sequence length distribution, as depicted in Fig. 3. Comparing T2V datasets with commonly used LLM corpora highlights three salient characteristics of video workloads that challenge conventional scheduling assumptions: *right-shifted*, *concentrated* and *discrete*.

- *Right-shifted.* The entire distribution is shifted dramatically to the right as a direct consequence of the data modality; after VAE encoding, even short video clips generate thousands of tokens, making the shortest T2V sequences more computationally demanding than the vast majority of LLM sequences (e.g., the 99.5% shorter than 8K tokens).
- *Concentrated.* Unlike the “flat” distributions of LLM corpora, T2V workloads exhibit “tall”, “narrow” profiles, indicating a high probability density within a specific band. This low variance signifies that most sequences are clustered tightly within a high-cost computational region, leading to a sustained and predictable load.
- *Discrete.* Since sequence length increases in discrete increments determined by video frame counts, marginal variations in video duration can lead to substantial computational disparities. Such discontinuity is a major source of inefficiency that amplifies the load imbalance across pre-defined training buckets.

Observation 2: Computation as the bottleneck and the limits of LLM-style scheduling.

A second defining characteristic of the T2V workloads is their extreme computational intensity, stemming from the DiT architecture’s full self-attention over a unified sequence of 3D patches of length L , which incurs quadratic ($O(L^2)$) complexity. When applied to the inherently long sequences described above, this operation becomes exceedingly *compute-intensive*, emerging as the dominant performance bottleneck.

Clearly, this characteristic limits the applicability of many approaches derived from the LLM domain [12, 55]. Existing LLM training frameworks are typically designed for text corpora with high variability in sequence length, where communication overhead across short sequences is the primary performance concern. However, these communication-oriented approaches may be insufficient for compute-bound workloads observed in T2V.

Example 1. Continuing the example in Fig. 2, we apply the communication-oriented dynamic parallelism method introduced in [55], which increases parallelism for the 113-frames sequence to $SP=8$ while reducing it for the 13- and 37-frame sequences to $SP=2$, respectively. As shown in Fig. 4, such optimization yields only limited benefits (gray region): while the overall latency is reduced from 58.6s to 45.2s, the performance bottleneck merely shifts to the 105-frame bucket,

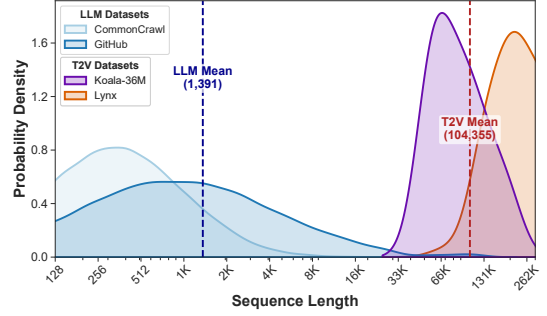


Fig. 3. Sequence length distributions for two T2V datasets (Koala [54] and an internal 1080p dataset called Lynx) and two LLM datasets (CommonCrawl and GitHub). The **vertical dashed lines** mark the average sequence lengths for each domain.

leaving substantial GPU idle times (i.e., 3.1s, 17.2s, and 30.1s). This example shows that applying communication-oriented optimizations in LLMs is challenging for T2V training due to the computation-intensive nature of the workload.

Motivation for Our Approach. Our analysis reveals that the interplay of heterogeneous video sequences and compute-intensive DiT architecture poses a unique training challenge for T2V models: one that existing parallelism approaches, including recent dynamic parallelism from the LLM field, are ill-equipped to solve. This inadequacy stems from two constraints that are inherent to all such approaches: (1) a *static spatial assignment*, which binds each data sample to a fixed set of GPUs for its entire lifecycle, and (2) a *strict temporal synchronization*, which mandates that all samples within an iteration launch simultaneously. Confronted with the profound heterogeneity of T2V data, these twin rigidities are the direct cause of the severe workload imbalance and resource under-utilization that plague current systems.

To overcome this rigidity, we introduce fine-grained spatio-temporal orchestration, a paradigm enabled by decomposing each sample’s computation into minimal, schedulable units (i.e., cascades). Such fine-grained decomposition unlocks independent control across both dimensions: on the temporal axis, we can precisely schedule each cascade’s launch time and assign it a bespoke parallel configuration to control its duration. On the spatial axis, we can map it onto a specific set of GPUs to create a communication-efficient physical layout. This flexible orchestration mitigates system idle time inherent in rigid schemes, thereby improving load balance and minimizing the end-to-end training makespan.

4 System Design

4.1 Overview

Arachne is a distributed training framework for T2V workloads by addressing the challenges outlined in §3. A high-level view of Arachne can be seen in Fig. 5, which illustrates its three main components: *Cascade-level parallelism planner*, *Topology-aware resource mapper*, and *Runtime executor*. We first overview each component and detail their design in the following subsections.

Cascade-level parallelism planner. The planner (illustrated in Fig. 5-①) is responsible for temporal orchestration, directly dismantling the temporal rigidity (i.e., a strict temporal synchronization) of traditional paradigms. It first decomposes each sample’s execution into a DAG of fine-grained, schedulable cascades. It then formulates the scheduling of these cascades as an optimization problem to minimize the global theoretical makespan, generating a logical execution plan that co-optimizes the launch time and parallel configuration for each cascade.

Topology-aware resource mapper. While the planner generates a temporally optimal logical plan, it does so on a homogeneous hardware abstraction. The mapper (illustrated in Fig. 5-②) addresses this by performing spatial orchestration to create a communication-efficient plan that breaks the spatial rigidity of fixed resource allocation. To achieve this, it refines the logical DAG via a two-level spatial arrangements: (1) At the intra-cascade level, it selects optimal physical GPUs to

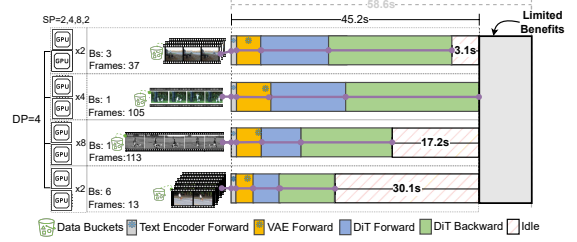


Fig. 4. Training step corresponding to Fig. 2, with parallelism **reconfigured** across buckets (SP=8 for the 113-frame bucket and SP=2 for the 13- and 37-frame buckets).

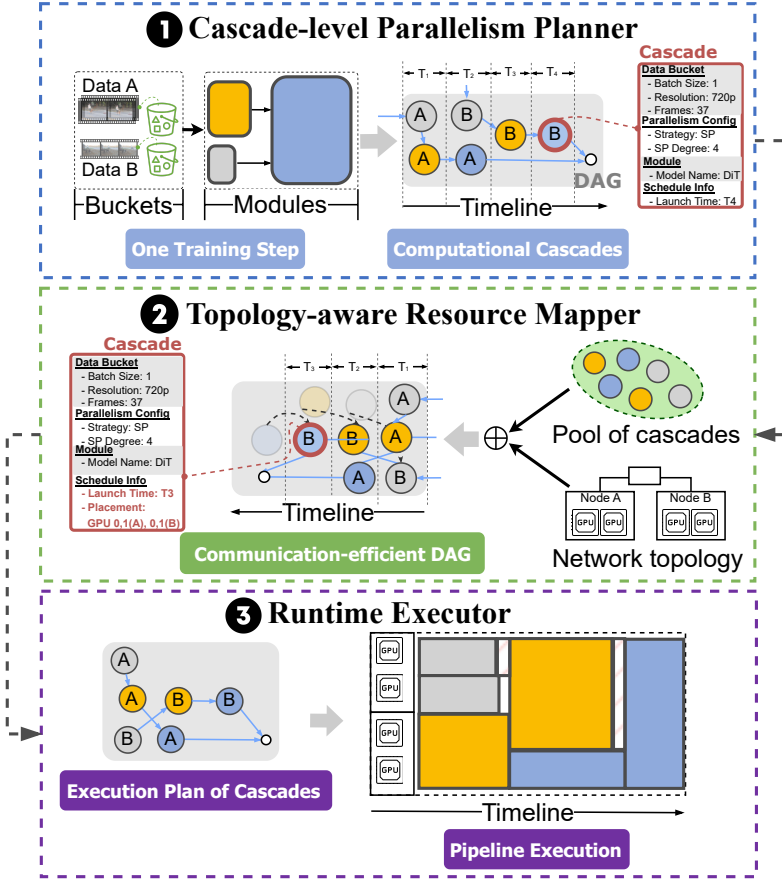


Fig. 5. Overview of Arachne.

minimize a cascade's internal communication. (2) At the inter-cascade level, it co-locates dependent cascades to maximize data locality and reduce transfer costs.

Runtime executor. Runtime executor (illustrated in Fig. 5-3) is the engine that enacts the complete spatio-temporal optimized execution plan crafted by the planner and mapper. It brings the optimized schedule to life by launching each cascade on its designated GPUs according to the precise timeline. Critically, the Executor manages the complex inter-cascade dependencies, handling state transitions (e.g., data handoffs) and synchronizations across different parallelism groups. To ensure correctness amidst this highly dynamic execution, it also implements a novel synchronization primitive for heterogeneous gradient accumulation, guaranteeing efficient and accurate gradient aggregation.

Example 2. Fig. 5 provides a toy example of training with Arachne on a cluster with 4 devices across 2 nodes. The training step takes two input data buckets, A and B, each of which is decomposed into three cascades corresponding to its constituent modules (e.g., text encoder, VAE, and DiT), resulting in six schedulable units. Cascade-level parallelism planner then first generates a DAG of cascades, assigning a launch time and a parallel configuration to each cascade. For example, the highlighted cascade (outlined in red) that associated with DiT and data B is scheduled at time slot T_4 with a SP=4. Subsequently, Topology-aware resource mapper ingests this DAG

Table 1. Nomenclature for the MILP Formulation.

Decision and Auxiliary Variables	
s_c	Launch time of cascade c
e_c	End time of cascade c
$x_{c,k}$	Binary variable: 1 if cascade c uses parallelism degree k , 0 otherwise
T	The makespan of the schedule (Objective to be minimized)
Sets and Indices	
I	Set of bucket samples i in the current batch
M	Set of model modules m (e.g., VAE)
C	Set of all cascades c , where each c corresponds to a pair (i, m)
K	Set of all available parallelism degrees k
A_t	Set of active cascades at time t , where $A_t = \{c \in C \mid s_c \leq t < e_c\}$
P_c	Set of direct predecessors of cascade c
Parameters	
$L_{c,k}$	Latency of cascade c with parallelism degree k
$\text{Mem}_{c,k}$	Peak memory requirement of cascade c with parallelism degree k
N	Total number of available GPUs
W	Maximum memory capacity of a single GPU

and the cluster’s network topology to find a physical mapping. By applying its intra-cascade and inter-cascade placement strategies to minimize communication overhead, it discovers opportunities to compact the schedule. This results in a rearranged timeline where the highlighted cascade is shifted from T_4 to T_3 . Concurrently, it assigns specific physical GPUs, mapping the cascade to a communicate-efficient configuration that communicates between nodes. Finally, Runtime executor takes the resulting optimized DAG and orchestrates the execution, realizing a fine-grained schedule that achieves efficient load balancing.

4.2 Cascade-level Parallelism Planner

A fundamental question for Arachne is how to determine the optimal execution strategy for a given training step. To this end, Cascade-level parallelism planner decomposes the training process into a set of basic schedulable units, referred to as cascades, and formulates their execution as a global *makespan minimization problem*, thereby deriving an optimized execution plan. In this section, we first provide a formal definition of the cascade and then present the corresponding optimization formulation.

DEFINITION 1. (Cascade) A cascade is the minimal, self-contained computational unit that can be directly executed within existing distributed training frameworks. Each cascade defines the configuration required for distributed execution, including input data, module specifications, parallelization strategy, GPU allocation, and scheduling parameters.

Example 3. Consider the case depicted in Fig. 5-①. The cascade is characterized by: (1) *input data*: one 37-frame 720p video clip; (2) *model specification*: DiT; (3) *parallelization strategy*: SP degree of 4; (4) *GPU allocation*: GPU 0 and 1 on both node A and node B; and (5) *launch time*: T_3 .

Problem Formulation. We formalize cascade scheduling as a makespan minimization problem, aiming to determine *when* each cascade executes and *how* much parallelism it uses to minimize

overall execution time. To focus on the intrinsic scheduling complexity, we treat GPUs as a homogeneous resource pool and intentionally abstract away concrete GPU placement; the resulting formulation therefore captures a logical scheduling optimum that jointly optimizes execution order and parallelism under dependency, memory, and aggregate resource constraints.

The primary input is a dependency graph constructed from each training batch, where nodes represent cascades and edges encode data dependencies. Based on this abstraction, we model the scheduling problem as a *Mixed-Integer Linear Programming* (MILP) formulation, with notation summarized in Table 1. The objective is to minimize the total makespan (T) by jointly optimizing each cascade's launch time s_c and parallel configuration $x_{c,k}$. The formulation enforces fundamental constraints on unique parallelism selection, precedence ordering, and per-GPU memory limits (Eqs. (4)–(6)), as well as an event-based resource constraint that bounds the number of concurrently active GPUs (Eq. (7)).

$$\text{Minimize } T \quad (1)$$

subject to:

$$e_c = \sum_{k \in K} (L_{c,k} \cdot x_{c,k}) + s_c, \quad \forall c \in C. \quad (2)$$

$$T \geq e_c, \quad \forall c \in C. \quad (3)$$

$$s_c \geq e_p, \quad \forall p \in P_c, \forall c \in C. \quad (4)$$

$$\sum_{k \in K} x_{c,k} = 1, \quad \forall c \in C. \quad (5)$$

$$\sum_{k \in K} (\text{Mem}_{c,k} \cdot x_{c,k}) \leq W, \quad \forall c \in C. \quad (6)$$

$$\sum_{c \in A_t} \left(\sum_{k \in K} k \cdot x_{c,k} \right) \leq N, \quad \forall t \geq 0. \quad (7)$$

The efficacy of our MILP formulation (§4.2) requires accurate apriori performance estimates. To this end, we developed two cost models² to predict the execution latency $L_{c,k}$ and peak GPU memory $\text{Mem}_{c,k}$ for each cascade c with any potential parallelism degree k .

Cost Model for VAE. The VAE's performance is dominated by its 3D convolutional layers. For long-duration or high-resolution videos, the intermediate activation tensors can become prohibitively large. The VAE architecture mitigates this by employing a tiling strategy that decomposes the input video into smaller, volumetric sub-tiles that are processed independently. Consequently, parallelism for the VAE is achieved at the tile level by distributing these sub-tiles across different GPUs. While this strategy effectively prevents OOM issues, it also introduces a complex performance prediction challenge.

To address this, we propose a hierarchical model that synergizes lightweight simulation with machine learning. A top-level simulator estimates the end-to-end latency for a VAE task with parallelism degree k . It operates by partitioning the input into standardized “base tiles”, simulating a round-robin distribution among the k workers, and using the completion time of the most loaded worker as the final latency. The simulator's accuracy hinges on the latency prediction for each base tile, which is provided by a two-stage hybrid model. This model first establishes an analytical

²We exclude the cost model for the text encoder in T2V model, as its contribution to the total makespan is negligible compared to the dominant bottleneck of processing long video sequences. This is due to the typically short text sequences in T2V training (e.g. under 256 tokens).

baseline via linear regression on the tile's volume. An XGBoost [8] model then refines this prediction by fitting the residuals, critically using a one-hot encoding of the tile's spatial shape (e.g., 256×256) as a feature to correct for performance variations arising from different data layouts.

Cost Model for DiT. The cost model for the Transformer-based DiT module provides the peak memory and latency estimates that are essential for the planner to find a memory-feasible schedule with minimal makespan.

The peak memory model avoids OOM errors by summing a fixed component for model states and a variable component for activations. The memory for model states ($\text{Mem}_{\text{states}}$), which depends on the total device count N and the applied ZeRO stage, is sharded and remains constant per GPU regardless of the sequence parallelism degree k . In contrast, the activation memory, determined by the total tokens ($\text{BatchSize} \cdot \text{SeqLen}$) and the empirically profiled per-token activation memory ($\text{Mem}_{\text{token}}$), is distributed across k GPUs. This leads to the per-GPU peak memory estimate:

$$\text{Mem}_{\text{DiT}}(\text{BatchSize}, \text{SeqLen}, k) = \text{Mem}_{\text{states}} + \frac{\text{BatchSize} \cdot \text{SeqLen} \cdot \text{Mem}_{\text{token}}}{k}. \quad (8)$$

The latency model decomposes the total execution time into two parts: parallelizable computation and communication. The final latency model combines these effects into a single equation:

$$\text{L}_{\text{DiT}}(\text{BatchSize}, \text{SeqLen}, k) = \frac{\alpha_1(\text{BatchSize} \cdot \text{SeqLen}) + \alpha_2(\text{BatchSize} \cdot \text{SeqLen}^2)}{k} + \frac{V_{\text{comm}}}{\text{BW}_{\text{eff}}(k)}, \quad (9)$$

where the first term models computation cost (coefficients α_1, α_2 from regression) and the second models communication cost, where V_{comm} denotes the communication volume and $\text{BW}_{\text{eff}}(k)$ is an effective bandwidth that selects between empirically measured intra-node and inter-node values based on the parallelism degree k .

Algorithm 1 Genetic Algorithm for Cascade Scheduling

- 1: **Input:** cascades C , cost model $\mathcal{M}$, and number of GPUs N
 - 2: **Hyper-parameters:** population size p , generations G , mutation rate μ , and elitism size ϵ
 - 3: **Output:** best makespan T^* and best schedule S^*
 - 4: $T^* \leftarrow \infty, S^* \leftarrow \text{None}$ ▷ Initialize the best-found solution
 - 5: $\mathcal{P} \leftarrow \text{INITIALIZEPOPULATION}(C, \mathcal{M}, p)$ ▷ Initialize a population $\mathcal{P}$ of p chromosomes
 - 6: **for** $g = 1 \rightarrow G$ **do**
 - 7: $\{S_i\}, \{T_i\} \leftarrow \text{EVALUATEFITNESS}(\mathcal{P}, C, \mathcal{M}, N)$ ▷ Find a short makespan
 - 8: $T_{\text{gen}}, S_{\text{gen}} \leftarrow \text{FINDBESTINGENERATION}(\{S_i\}, \{T_i\})$
 - 9: **if** $T_{\text{gen}} < T^*$ **then** ▷ Update the global best solution
 - 10: $T^* \leftarrow T_{\text{gen}}$
 - 11: $S^* \leftarrow S_{\text{gen}}$
 - 12: **end if**
 - 13: $\mathcal{P} \leftarrow \text{EVOLVENEXTGENERATION}(\mathcal{P}, \{T_i\}, \mu, \epsilon)$ ▷ Via elitism, crossover, and mutation
 - 14: **end for**
 - 15: **return** T^*, S^* ▷ Return the best schedule
-

Genetic Algorithm for Heuristic Cascade Scheduling. While the MILP formulation in §4.2 defines the theoretical optimum, the NP-hard nature of the problem makes exact solutions impractical for real-time scheduling. We therefore adopt a *genetic algorithm* (GA) as an anytime heuristic to approximate the logical optimum under a bounded planning budget, which can achieve a favorable trade-off between planning latency and schedule quality (see §5.6 for a detailed discussion).

Specifically, the GA decouples parallel configuration selection from launch-time scheduling. Each chromosome encodes a complete assignment of parallelism degrees $\{x_{c,k}\}$ for all cascades (as shown in Algorithm 1, line 5), and its fitness is evaluated by a fast list scheduler that heuristically determines the launch times $\{s_c\}$ and resulting makespan by prioritizing cascades on the critical path (lines 6–7), in a topology-agnostic manner assuming a homogeneous GPU resource pool. Across generations, the population is iteratively evolved through elitism, crossover, and mutation (line 13), enabling efficient exploration of the parallelism configuration space and progressive refinement toward lower makespans.

4.3 Topology-aware Resource Mapper

Cascade-level parallelism planner treats GPUs as a homogeneous resource pool, which can result in initial allocations with substantial communication overhead. To mitigate this, Topology-aware resource mapper incorporates physical hardware topology during cascade placement, completing the planner's logical schedule into a topology-aware, end-to-end execution plan for efficient T2V training.

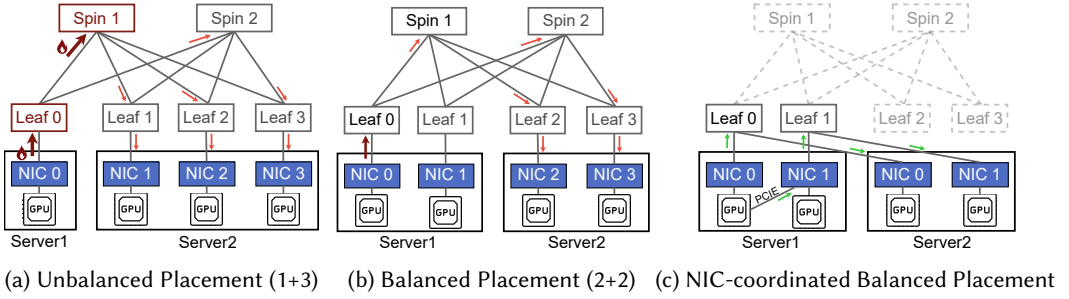


Fig. 6. Comparison of three different resource placement strategies

Intra-cascade Placement. Topology-aware resource mapper prioritizes the communication patterns in a single cascade for resource assignments. This is achieved by using a *four-level hierarchical prioritization scheme* that minimizes communication latency and prevents network bottlenecks.

- *Priority 1 (Within-Node Placement).* The highest priority is to place an entire cascade within a single compute node. That is, if a node has enough available GPUs to satisfy its parallelism degree, Topology-aware resource mapper restricts placement to that node. Such localization ensures that collective communication operations (e.g., All-Reduce, All-to-All) are confined to high-bandwidth, low-latency intra-node interconnects such as NVLink.
- *Priority 2 (Balanced Cross-Node Placement).* When within-node placement is infeasible, the priority is to distribute the cascade crossing nodes in a balanced manner. By balanced, we refer to an even distribution of inter-node communication that helps prevent localized congestion and reduces the likelihood of network hotspots.

Example 4. For a cascade with a parallelism degree of 4 (SP=4), the asymmetric 1+3 placement in Fig. 6 (a) concentrates all traffic through a single data path, creating a severe bottleneck on the node's PCIe bus, its NIC, and the corresponding switch uplink (indicated by the fire symbol). In contrast, the symmetric 2+2 placement in Fig. 6 (b) evenly distributes communication across NICs (balanced), mitigating congestion and delivering better performance.

- *Priority 3 (NIC-Coordinated Placement).* Given a balanced inter-node placement (e.g., a 2+2 distribution), the third priority is to select specific GPUs that optimize data paths across the network

fabric. This strategy leverages a common *multi-rail cluster architecture* where same-numbered NICs across different nodes are connected to the same leaf switch.

Example 5. Fig. 6 (c) illustrates the placement in which NIC indices are aligned across nodes (0, 1 on both), confining all inter-node communication to a single leaf switch (e.g., data from GPU 0 (node A) to GPU 1 (node B) travels via the local PCIe bus to NIC 1 (node A), through leaf switch 1, reaches NIC 1 (node B), and finally arrives at GPU 1 (node B), completely avoiding the high-latency spine switch). The balanced placement in Fig. 6 (b), however, assigns node A's GPUs to NICs 0, 1 and node B's to 2, 3, forcing all inter-node traffic through the spine switch.

- *Priority 4 (Internal Communication-Aware Placement).* On modern GPU servers, latency between a GPU and a given NIC may vary due to traversal across PCIe switches or NUMA domains. In cases where multiple GPU combinations achieve equivalent NIC coordination, this strategy considers the topology of intra-node communication and selects the combination that minimizes the cumulative communication distance to the assigned NICs, thereby reducing intra-node latency and ensuring the most efficient path to the inter-node network.

Inter-cascade Placement. For interdependent cascades (e.g., a VAE passing its output to a DiT), we co-design placement with the data handoff mechanism introduced in Runtime executor (§4.4) to enable efficient transfers. Topology-aware resource mapper follows a prioritized approach to achieve this: Its primary objective is to maximize the physical GPU overlap between the producer and consumer. A high degree of overlap allows Runtime executor to perform a highly efficient in-place handoff, where the common GPUs act as a distributed cache and eliminate the need for explicit data movement. If such an overlap is infeasible, the mapper pursues a secondary objective: optimizing the handoff engine in Runtime executor. To do so, it attempts to co-locate the main rank of the producer or consumer with the central broker rank, thereby confining the Gloo-based data transfers to high-speed intra-node interconnects and minimizing overhead for unavoidable data movement.

4.4 Runtime Executor

Once it receives the optimized DAG from Topology-aware resource mapper, the primary role of Runtime executor is to translate this static plan into a live, dynamic execution on the cluster. It achieves this by treating each GPU as a dynamic resource that is rapidly re-tasked according to the DAG's spatio-temporal schedule. For instance, a single GPU might process a VAE cascade for one data sample and then be immediately re-assigned to a DiT cascade for another. To manage this fluidity, the Executor is responsible for two critical functions detailed below: orchestrating efficient inter-cascade data handoffs and resolving the complex challenge of heterogeneous gradient accumulation.

Optimized Inter-cascade Data Handoff. Runtime executor's key function in managing inter-cascade data dependencies (e.g., passing a latent tensor from a VAE to a DiT) is an adaptive data handoff mechanism. This mechanism avoids the prohibitive memory footprint of naively buffering tensors globally by leveraging the physical locality of the producer and consumer GPU groups, as determined by Topology-aware resource mapper.

The mechanism operates in two modes based on this locality. First, for overlapping GPU groups, it performs a highly efficient in-place handoff, where common GPUs act as a distributed cache. If the consumer group is a superset, a localized broadcast from these caching GPUs efficiently updates the new peers. Second, for disjoint groups, a decoupled asynchronous engine allows the producer to "fire-and-forget" its output tensor, offloading it to a central broker rank via the Gloo

backend. This use of Gloo establishes a dedicated channel for the handoff, ensuring it does not contend with performance-critical NCCL collectives like All-Reduce. The consumer retrieves the tensor via a get request when ready, enabling the entire transfer to seamlessly overlap with ongoing computation. This adaptive strategy ensures data is transferred with minimal overhead, precisely tailored to the dynamic placement of the cascades.

Heterogeneous Gradient Accumulation. A key challenge arising from Runtime executor’s dynamic cascade dispatch is heterogeneous gradient accumulation. The DAG schedule breaks the traditional one-to-one GPU-to-sample-gradient mapping, leading to a complex gradient landscape. For instance, while processing four data chunks (A, B, C, D), one GPU subgroup might accumulate a composite gradient of $G_A + G_B$, a second subgroup could hold a different composite $G_C + G_D$, and a third might only contain a pure gradient G_A . This heterogeneity, where GPUs hold fundamentally different pre-summed gradient combinations, renders a conventional global All-Reduce mathematically incorrect, as it cannot disentangle the components.

To address this, Runtime executor introduces a two-phase representative-based decentralized synchronization strategy: A minimal set of representative GPUs covering all gradient components first performs an efficient All-Reduce to compute the global gradient, G_{final} . Then, each representative simultaneously broadcasts G_{final} to its local group. This decentralized approach efficiently handles large gradients, leverages NCCL, and ensures consistent updates.

5 Experiments

5.1 Experimental Setup

Hardware and Software Environment. All experiments are conducted on a cluster with 64 NVIDIA H100 (80GB) GPUs. Each 8-GPU node is equipped with high-bandwidth NVLink for intra-node communication and eight 400 Gbps InfiniBand NICs for inter-node connectivity. Our software stack consists of PyTorch v2.7.1 and CUDA v12.6. The end-to-end performance results (§5.2) are reported on 16 GPUs (2 nodes), while the scalability evaluation (§5.3) is conducted on 64 GPUs.

Baselines. We compare Arachne with two state-of-the-art distributed training systems, namely **Megatron-LM** [44] and **DeepSpeed** [40], as well as **FlexSP** [55], a representative LLM training system optimized for variable-length corpora. Here, Megatron-LM serves as a general-purpose baseline featuring static parallelism, combining DP (with ZeRO-1 optimizations) and CP, and DeepSpeed integrates the ZeRO optimizer with Ulysses-style SP; both rely on fixed parallelism configurations (e.g., DP=4, SP=4) that are manually tuned for optimal efficiency. FlexSP dynamically adjusts the parallelism degree at each iteration based on workload variations, which is configured to select different SP degrees for samples drawn from different buckets on a per-iteration basis.

Experimental Workloads. To evaluate the training performance under realistic conditions, we adopt the public datasets used in Open-Sora [60] and conduct experiments using the three-stage training strategy described in §2.2, where each stage progressively increases data scale, resolution, and computational complexity:

- **Stage 1 (Base Training).** The initial workload uses the WebVid dataset [2], comprising approximately 7 million video clips (~23,000 hours) at a 360p resolution.
- **Stage 2 (Quality Improving).** The second stage scales up to Koala [54], containing about 20 million clips (~100,000 hours) at a higher 720p resolution.
- **Stage 3 (High-Fidelity Fine-tuning).** The final stage uses a large-scale internal dataset named Lynx that contains approximately 10 million clips (~55,000 hours) at 1080p to represent a production-level workload.

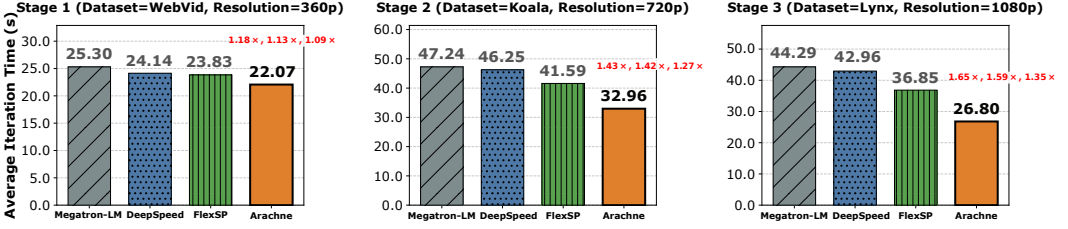


Fig. 7. **Average iteration time** (in seconds) across three training stages for the HunyuanVideo model. The annotations above the Arachne bars indicate relative speedups compared to the baseline systems.

Experimental Models. We evaluate our system on three T2V models (ranging from 1.3B to 13B parameters) to demonstrate the generality across scales and architectures of Arachne. Table 2 summarizes the details of these models.

- **Wan2.1** [52]. We use the 1.3B variant of Wan2.1 as our small-scale model, a popular open-source T2V model employing both self-attention and text-video cross-attention.
- **CogVideoX** [56]. For the medium-scale model, we use the 5B variant of CogVideoX, a pioneering T2V model that established the use of 3D full attention.
- **HunyuanVideo** [23]. We adopt HunyuanVideo (13B), a large-scale model featuring a hybrid dual- and single-stream architecture.

Table 2. Architectural details of the T2V models evaluated.

T2V Models	#Parameters	#Layers	#Attn Heads	Hidden Dims
Wan2.1	1.3B	30	12	1536
CogVideoX	5B	30	30	1920
HunyuanVideo	13B	60	24	3072

Protocols. To ensure a fair comparison, we apply a unified bucket configuration coupled with adaptive batching across all systems. We cap the maximum sequence length at 113 frames as a practical trade-off to maximize data coverage while generally avoiding OOM errors for the baselines. Activation checkpointing is universally enabled to reduce memory consumption. In contrast to the baselines’ static configurations, Arachne performs fine-grained spatio-temporal orchestration at each training iteration, dynamically generating and executing an optimized plan for the current workload’s cascades. For all experiments, we measure performance over 50 iterations following a 10-iteration warm-up to ensure stability.

5.2 End-to-End Performance

Overall Results. The main results for the training of HunyuanVideo are presented in Fig. 7. Overall, Arachne consistently outperforms all baselines across all three stages, with its performance advantage becoming increasingly pronounced from Stage 1 (360p) to Stage 3 (1080p). As data heterogeneity and computational density intensify at higher resolutions, the resulting workload imbalance exacerbates, thereby amplifying the benefits of Arachne’s fine-grained execution. **One remarkable outcome is that Arachne achieves peak speedups of 65% over Megatron-LM, 59% over DeepSpeed, and 35% over FlexSP in Stage 3.**

It is worth noting that the absolute average iteration time in Stage 3 (26.80s) is “counterintuitively” lower than that in Stage 2 (32.96s). This observation actually stems from hardware memory

constraints at 1080p, which cap the maximum sequence length at 57 frames and thus reduce overall computation by skewing the workload distribution toward shorter sequences. Nevertheless, the computational disparity across buckets with different frame counts is substantially larger in Stage 3 (1080p), leading to more severe workload imbalance under static or coarse-grained parallelization. Notably, by effectively mitigating this amplified imbalance through fine-grained spatio-temporal orchestration, Arachne is able to capitalize on the increased heterogeneity in Stage 3, yielding its largest relative performance gains in this setting.

On the other hand, we analyze the average GPU idle ratio—defined as the proportion of total training time that GPUs spend in an idle state—as shown in Fig. 8. For Megatron-LM and DeepSpeed, the idle ratio increases progressively across the three stages, confirming that workload heterogeneity exacerbates as the training scales to higher resolutions. FlexSP partially alleviates this issue by dynamically forming heterogeneous sequence parallelism groups based on the sequence length of each iteration; however, it still exhibits substantial idle time, which rises from 23.5% in Stage 1 to 33.6% in Stage 3. In stark contrast, Arachne leverages its fine-grained spatio-temporal orchestration to drastically compress idle time, achieving significantly lower ratios of 12.1%, 9.0%, and 8.1%, respectively. This inverse trend demonstrates that increased workload heterogeneity actually favors Arachne, as the greater data diversity provides a more flexible scheduling space for orchestrating cascades, ultimately yielding superior workload balance and hardware utilization.

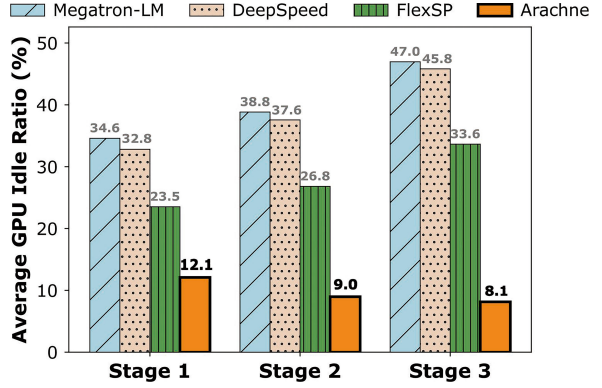


Fig. 8. Average GPU Idle Ratio on three training stages.

5.3 Scalability Evaluation

To evaluate the robustness and scalability of Arachne, we conduct experiments across three dimensions of large-scale training: *computational intensity*, *workload heterogeneity*, and *cluster size*. Our exploration centers around the following three key questions:

- (i) How does Arachne perform as the training task becomes increasingly compute-intensive?
- (ii) To what extent can Arachne sustain efficiency under diverse and imbalanced workloads?
- (iii) How well does Arachne scale when deployed over larger clusters?

Model-size Scalability. We evaluate the scalability of Arachne with computational intensity by benchmarking it on three T2V models of increasing size: Wan2.1-1.3B, CogVideoX-5B, and HunyuanVideo-13B. To isolate the effect of model scale, we fix the workload to a moderately intensive Stage 2 configuration (i.e., Koala at 720p).

Fig. 9 shows that Arachne’s performance advantage consistently amplifies as model size increases. Compared to Megatron-LM, Arachne improves throughput by 35% on the 1.3B model and by 43% on the 13B model. This widening gap reflects the fact that larger models are more compute-intensive, which magnifies the penalty of workload imbalance. While such an imbalance increasingly constrains static baselines, Arachne effectively converts the resulting idle time into productive computation. Notably, the performance gap between Arachne and FlexSP also

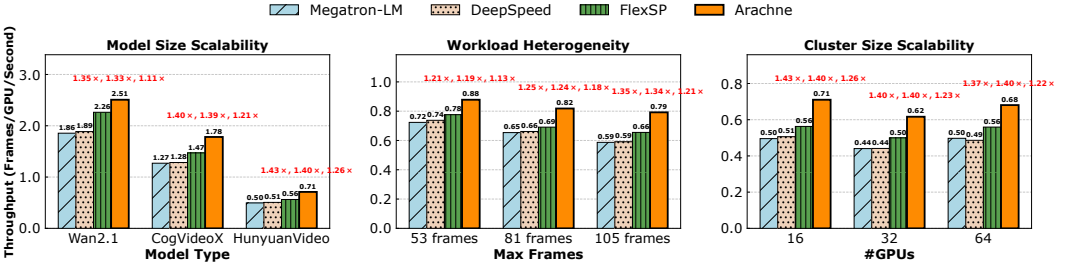


Fig. 9. **Throughput scalability evaluation** of Arachne under increasing training complexity, evaluated on HunyuanVideo. The figure evaluates scalability along **three dimensions**: *model size*, *workload heterogeneity* induced by larger maximum frame windows, and *cluster size*.

widens with model size, indicating that coarse-grained parallelism adaptation alone becomes insufficient as compute-induced workload imbalance intensifies. Together, these results demonstrate that Arachne not only scales with computational intensity, but increasingly outperforms both static and communication-oriented baselines as T2V models grow larger, underscoring its strong scalability for *future, even larger models*.

Workload-Heterogeneity Scalability. Next, we vary the degree of data imbalance by expanding the maximum frame window, evaluating three configurations: 53, 81, and 105 frames. These frame caps create workloads of increasing heterogeneity by drawing samples from progressively larger and thus more imbalanced portions of the full dataset distribution illustrated in Fig. 3 (10.8%, 46.9%, and 70.4%, respectively). To ensure that the observed effects can be attributed solely to workload heterogeneity, we keep the model fixed (HunyuanVideo) and adopt the Stage 2 setup.

The results, presented in Fig. 9, show that Arachne’s performance advantage amplifies as workload heterogeneity increases. For instance, its throughput improvement over Megatron-LM grows from a modest 21% in the most homogeneous setting (53-frame window) to 35% in the most imbalanced setting (105-frame window). A similar widening gap is observed when comparing against FlexSP, indicating that Arachne continues to benefit from increased data heterogeneity beyond what can be addressed by coarse-grained parallelism adaptation. This trend arises because greater data heterogeneity induces more severe workload imbalance, which in turn provides greater opportunities for Arachne’s fine-grained orchestration. Overall, these results indicate that Arachne’s ability to mitigate load imbalance strengthens as data heterogeneity intensifies, highlighting its suitability as a scalable solution for increasingly *large and diverse real-world datasets*.

Cluster Scalability. Finally, we evaluate Arachne across GPU clusters of 16, 32, and 64 GPUs using the large-scale HunyuanVideo model under a fixed Stage 2 configuration, in order to assess its scalability as the cluster size increases.

The results in Fig. 9 demonstrate Arachne’s stable cluster scalability. Its throughput advantage over both baselines remains consistently high, ranging between 30% and 40% across all evaluated scales from 16 to 64 GPUs. As the cluster scales up, the number of cascades within each iteration grows alongside the available GPUs. The expanded resource pool in the spatial dimension provides Arachne with greater flexibility in placing and scheduling cascades across devices, enabling it to effectively absorb workload imbalance at larger scales. These results show that Arachne maintains stable performance as the cluster grows, highlighting its robustness and practicality for *large-scale production training environments*.

In summary, experiments across model size, workload heterogeneity, and cluster size demonstrate that Arachne exhibits a clear scaling trend, consistently achieving efficiency gains with

large-scale training scenarios. **This positions Arachne as a robust and future-ready system for large-scale T2V training under ever-expanding computational and data demands.**

5.4 Ablation Study

To validate the effectiveness of the resource placement strategies introduced in Arachne for minimizing communication overhead, we conduct an ablation study of Topology-aware resource mapper. All configurations are benchmarked on HunyuanVideo with three-stage training workloads from our end-to-end evaluation (§5.2). Table 3 presents the results.

Table 3. Ablation study on HunyuanVideo, showing the average iteration time (in seconds), with the corresponding slowdown (%) shown in parentheses. “w/o Mapper” denotes removing Topology-aware resource mapper entirely, while “w/o Inter-cascade” and “w/o Intra-cascade” partially disable the inter- and intra-cascade placement optimization, respectively.

Method	Stage 1	Stage 2	Stage 3
Arachne	22.07	32.96	26.80
w/o Inter-cascade	22.98 (+4.13%)	34.29 (+4.04%)	27.78 (+3.64%)
w/o Intra-cascade	23.76 (+7.68%)	35.76 (+8.48%)	30.41 (+13.67%)
w/o Mapper	23.89 (+8.26%)	37.09 (+12.53%)	31.89 (+19.26%)

The results show that disabling Topology-aware resource mapper significantly increases iteration times across all stages, with the most pronounced impact in Stage 3, which slows down by a substantial 19.26%. A finer-grained analysis further reveals a clear hierarchy within the mapper’s components. The intra-cascade placement strategy is the dominant contributor; disabling it in Stage 3 causes a 13.67% slowdown, far greater than the 3.64% slowdown from disabling the inter-cascade strategy. These findings validate our mapper’s design, particularly its emphasis on optimizing intra-cascade communication. More fundamentally, they demonstrate that an optimized temporal plan is only truly effective when coupled with topology-aware spatial orchestration, confirming the necessity of our holistic spatio-temporal orchestration paradigm.

5.5 Case Study

To analyze how Arachne achieves its performance gains, we conduct a detailed case study on HunyuanVideo using Stage 2 workloads, combining execution timeline visualization and workload balance analysis.

Execution Timeline Visualization We revisit the example introduced in Fig. 4 to provide a before-and-after comparison against the execution schedule generated by Arachne. As shown in Fig. 10, Arachne produces an optimized spatio-temporal execution by exercising fine-grained control over both temporal scheduling and spatial allocation of cascades, breaking the rigid constraints imposed by prior schedulers. Temporally, independent cascades are staggered to fill idle compute periods: shorter cascades from data A and B are dispatched to overlap with longer-running ones. Spatially, Arachne decouples cascades from fixed GPU sets to enable dynamic resource allocation. This behavior is exemplified by data D, whose text encoder, VAE, and DiT cascades are dynamically assigned 4 GPUs (12--15), 6 GPUs (8--13), and 12 GPUs (0--11), respectively. As a result, the makespan for

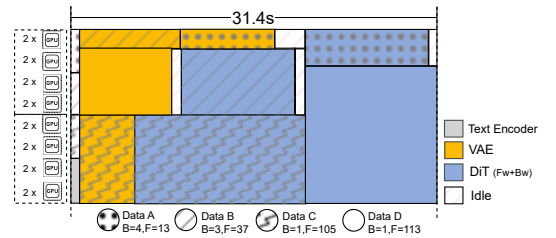


Fig. 10. Execution timeline visualization in case study.

this iteration is reduced to 31.4s, representing a 30.5% reduction from the 45.2s FlexSP baseline (as shown in Fig. 4).

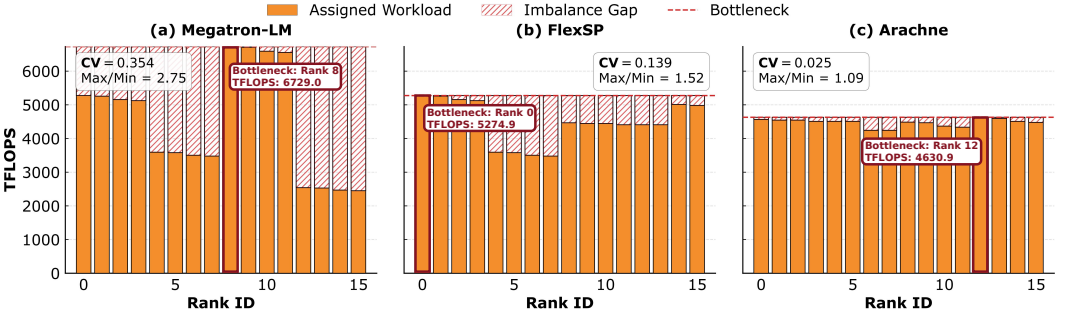


Fig. 11. **Per-rank TFLOPS distribution** in a training iteration, where **hatched regions** highlight under-utilized computation relative to the bottleneck. We use *Coefficient of Variation* (CV), defined as the standard deviation divided by the mean of per-rank TFLOPS, to quantify workload imbalance across ranks. Megatron-LM is used as the representative baseline for static-parallelism and excludes DeepSpeed for simplicity.

Workload Balancing Analysis. To attribute the observed performance gains to improved workload balancing, we analyze both the *spatial distribution* and *temporal stability* of per-rank computation. Fig. 11 presents the per-rank TFLOPS distribution within a single training iteration. Compared to all baselines, Arachne achieves a more balanced workload across GPU ranks, as evidenced by both a lower CV and smaller imbalance gaps. Fig. 12, on the other hand, measures the CV of per-rank TFLOPS over 50 consecutive training iterations. Arachne consistently maintains a low CV, remaining around 0.10 across iterations, whereas the baselines exhibit substantially higher variance over time. Taken together, these results demonstrate that Arachne’s fine-grained spatio-temporal orchestration yields a workload allocation that is both spatially balanced and temporally stable, underpinning its sustained training throughput improvements.

5.6 Complexity and Scalability of Arachne Planner

We evaluate the complexity and scalability of Arachne’s planning pipeline to assess its practicality for runtime deployment, and demonstrate that (i) the planning latency time can be amortized and overlapped with training execution as cluster scale increases, and (ii) high-quality schedules can be obtained under a bounded budget via anytime planning from GA. In our setting, the dominant drivers of planning complexity are the number of GPUs and cascades, which jointly determine the search space for scheduling and parallelism decisions. In this section, we measure the end-to-end latency of Arachne’s two-stage planning pipeline (i.e., the planner first produces a topology-agnostic logical plan, and then the mapper refines it into a topology-aware physical plan) and compare against placement-aware optimal (best-known) schedules.

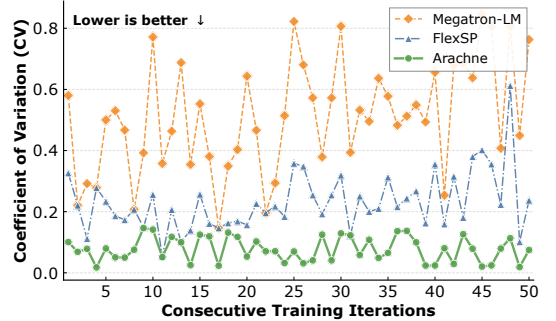
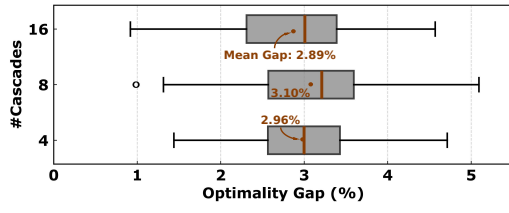
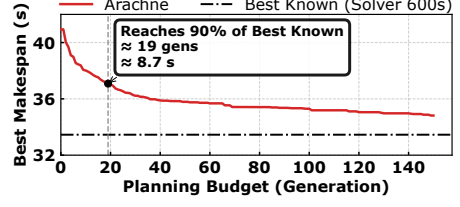


Fig. 12. **Temporal stability** measured by CV of per-rank TFLOPS over 50 consecutive training iterations.



(a) Box plot of **normalized optimality gaps** relative to the known optimum across different numbers of cascades counts in an 8-GPU setup.



(b) Best-so-far makespan versus planning budget (generations) in a 32-GPU, 16-cascade setting, compared to the best-known solution (600 s solver).

Fig. 14. Planning quality and efficiency of Arachne.

Planning Overhead under GPU Scaling. Fig. 13 compares the average planning latency with the training makespan. As expected, planning time scales with the number of GPUs and cascades due to the expanding search space. In large-scale configurations, the raw planning latency (shown by the orange curve) may exceed the execution time of a single iteration.

Importantly, planning does not directly stall training execution. Plan generation runs on host CPUs and is decoupled from GPU execution, allowing planning to proceed asynchronously with iterations. As cluster scale increases, the availability of additional CPU cores enables parallel planning for multiple future iterations, so that execution plans can be generated several steps ahead. This naturally forms a *producer-consumer pipeline* between planning and execution. As a result, planning latency can be amortized and overlapped with GPU training, rendering its impact negligible on overall training throughput.

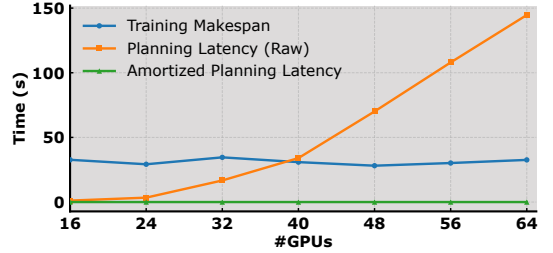


Fig. 13. Per-iteration training makespan, raw planning latency, and **amortized planning latency** (overlapped with execution) of Arachne's Planner.

Anytime Planning under Combinatorial Explosion. Although planning latency can be overlapped with execution, the quality of the resulting schedule remains a key determinant of hardware utilization efficiency and overall training throughput. Runtime orchestration must jointly optimize parallelism configurations, GPU mappings, and execution timelines, giving rise to an NP-hard combinatorial search space that renders exact solvers impractical for online use due to their prohibitively long convergence times. We therefore evaluate Arachne's solution quality under a limited planning budget.

Fig. 14 (a) presents the results in a relative small-scale (i.e., 8-GPU) setting. Unsurprisingly, the genetic planner consistently maintains optimality gaps within 5% across 4, 8, and 16 cascades, indicating stable and near-optimal scheduling performance. Motivated by the stable performance observed in the small-scale setting, we further scale the evaluation to a more realistic setting (32-GPU, 16-cascade). As illustrated in Fig. 14 (b), the planner rapidly identifies a high-quality schedule, reaching within 90% of the best-known solution in only 8.7 seconds (19 generations), after which additional planning yields diminishing returns beyond 50 generations. The results show that Arachne achieves both robust solution quality and efficient anytime planning, making genetic-based orchestration practical for runtime deployment under severe combinatorial explosion.

6 Related Works

Dynamic and Heterogeneity-Adaptive Systems. Recent work in the LLM domain has identified data heterogeneity during training as a key performance challenge, motivating dynamic training frameworks that primarily reduce communication overhead from short text sequences [12, 55]. In contrast, T2V workloads exhibit fundamentally different characteristics: long, discretely distributed video sequences and the compute-intensive DiT architecture shift the training bottleneck from communication to computation. Arachne addresses this domain-specific challenge through fine-grained spatio-temporal orchestration to mitigate workload imbalance induced by data heterogeneity in large-scale T2V training.

Long Sequence Training. A prominent line of work tackles the challenge of long sequence T2V training by modifying the attention mechanism itself, for instance, through sparse attention [48, 59] or elaborate parallel strategies like ring attention [4, 10, 26, 29, 32]. These methods are orthogonal to our work: they optimize the implementation of the core computation, whereas Arachne operates at a higher level by scheduling these computations as black-box cascades across a heterogeneous workload. Consequently, such optimized attention implementations can be seamlessly integrated into our framework to further enhance performance.

Heterogeneous Cluster Training. Prior work on heterogeneous cluster training focuses on adapting execution to differences in GPU compute capability, memory capacity, or interconnect bandwidth across devices [19, 35, 47, 50]. These systems primarily address hardware-induced heterogeneity and typically assume homogeneous workloads. In contrast, Arachne targets workload-induced heterogeneity in T2V training, which arises from highly various video resolutions and frame lengths. While our current design focuses on data-driven workload heterogeneity, the cascade-level orchestration in Arachne naturally provides a foundation for incorporating hardware heterogeneity as well, by extending the cost model to account for device-specific computation and communication characteristics.

Memory Optimization. A body of work reduces the memory footprint of large-scale model training through techniques such as activation recomputation [9, 22] and parameter or activation offloading to host memory or secondary storage [41, 42, 53]. These approaches alleviate GPU memory constraints at the cost of additional recomputation or data transfer overhead, and are largely agnostic to workload imbalance. Rather than modifying model semantics or memory access patterns, Arachne focuses on orchestrating heterogeneous T2V workloads through fine-grained spatio-temporal scheduling to improve execution efficiency. As such, Arachne is orthogonal to memory optimization techniques and can be naturally combined with them to relieve GPU memory pressure and enable training of larger-scale T2V models.

7 Conclusion

We presented Arachne, a novel distributed training framework for efficient T2V model training, based on the fine-grained spatio-temporal orchestration paradigm. Decomposing the training workload into fine-grained cascades, Arachne optimized their physical placement and execution timeline, addressing the severe workload imbalance caused by data heterogeneity. The comprehensive evaluation on leading T2V models showed that Arachne achieved up to 65% reduction in iteration time compared to state-of-the-art training frameworks. More importantly, it revealed that the performance advantage of this approach scales with both model complexity and workload heterogeneity, confirming its robustness and practicality as a solution for large-scale generative model training in the future.

References

- [1] Ivan Bacher, Hossein Javidnia, Soumyabrata Dev, Rahul Agrahari, Murhaf Hossari, Matthew Nicholson, Clare Conran, Jian Tang, Peng Song, David Corrigan, and François Pitié. An advert creation system for 3d product placements. In *Machine Learning and Knowledge Discovery in Databases: Applied Data Science Track - European Conference, ECML PKDD*, volume 12460 of *Lecture Notes in Computer Science*, pages 224–239. Springer, 2020.
- [2] Max Bain, Arsha Nagrani, Gül Varol, and Andrew Zisserman. Frozen in time: A joint video and image encoder for end-to-end retrieval. In *IEEE International Conference on Computer Vision*, 2021.
- [3] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani, and Robin Rombach. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023. License: CC BY, Stability AI.
- [4] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped Attention: Faster Ring Attention for Causal Transformers, 2023.
- [5] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. Technical report, OpenAI, February 2024.
- [6] Haoxuan Che, Xuanhua He, Quande Liu, Cheng Jin, and Hao Chen. GameGen-X: Interactive Open-world Game Video Generation. *arXiv preprint arXiv:2411.00769*, 2024.
- [7] Shoufa Chen, Chongjian Ge, Yuqi Zhang, Yida Zhang, Fengda Zhu, Hao Yang, Hongxiang Hao, Hui Wu, Zhichao Lai, Yifei Hu, Ting-Che Lin, Shilong Zhang, Fu Li, Chuan Li, Xing Wang, Yanghua Peng, Peize Sun, Ping Luo, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Xiaobing Liu. Goku: Flow Based Video Generative Foundation Models. *arXiv preprint arXiv:2502.04896*, 2025.
- [8] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [9] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv: Learning, arXiv: Learning*, Apr 2016.
- [10] Jiarui Fang and Shangchun Zhao. A unified sequence parallelism approach for long context generative ai. *CoRR*, abs/2405.07719, 2024. 12 pages. Subjects: Machine Learning (cs.LG); Artificial Intelligence (cs.AI). Code available at [thishttpsURL](https://github.com/jiarui-fang/long-context-parallelism).
- [11] Ruili Feng, Han Zhang, Zhantao Yang, Jie Xiao, Zhilei Shu, Zhiheng Liu, Andy Zheng, Yukun Huang, Yu Liu, and Hongyang Zhang. The Matrix: Infinite-Horizon World Generation with Real-Time Moving Control. *arXiv preprint arXiv:2412.03568*, 2024.
- [12] Hao Ge, Fangcheng Fu, Haoyang Li, Xuanyu Wang, Sheng Lin, Yujie Wang, Xiaonan Nie, Hailin Zhang, Xupeng Miao, and Bin Cui. Enabling Parallelism Hot Switching for Efficient Training of Large Language Models. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, pages 178–194. ACM, November 2024.
- [13] Lei Guan, Dong-Sheng Li, Jiye Liang, Wen-Jian Wang, Ke-shi Ge, and Xicheng Lu. Advances of pipeline model parallelism for deep learning training: An overview. *J. Comput. Sci. Technol.*, 39(3):567–584, 2024.
- [14] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Fei-Fei Li, Irfan Essa, Lu Jiang, and José Lezama. Photo-realistic video generation with diffusion models. In *ECCV*, volume 15137 of *Lecture Notes in Computer Science*, pages 393–411. Springer, 2024.
- [15] David Ha and Jürgen Schmidhuber. World models. *CoRR*, abs/1803.10122, 2018.
- [16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [17] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *NeurIPS*, pages 103–112, 2019.
- [18] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *CoRR*, abs/2309.14509, 2023.
- [19] Xianyan Jia, Le Jiang, Ang Wang, Wencong Xiao, Ziji Shi, Jie Zhang, Xinyuan Li, Langshi Chen, Yong Li, Zhen Zheng, Xiaoyong Liu, and Wei Lin. Whale: Efficient giant model training over heterogeneous gpus. In *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC 22)*, Carlsbad, CA, USA, July 2022. USENIX Association.
- [20] Xuan Ju, Yiming Gao, Zhaoyang Zhang, Ziyang Yuan, Xintao Wang, Ailing Zeng, Yu Xiong, Qiang Xu, and Ying Shan. Miradata: A large-scale video dataset with long durations and structured captions. In *NeurIPS*, 2024.
- [21] Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022.
- [22] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. *arXiv: Learning, arXiv: Learning*, Jun 2020.

- [23] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.
- [24] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *CoRR*, abs/2205.05198, 2022.
- [25] Mario Michael Krell, Matej Kosec, Sergio P. Perez, and Andrew W. Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. In *ICLR*, 2021.
- [26] Dacheng Li, Rulin Shao, Anze Xie, Eric P. Xing, Xuezhe Ma, Ion Stoica, Joseph E. Gonzalez, and Hao Zhang. DIST-FLASHATTN: Distributed Memory-efficient Attention for Long-context LLMs Training. In *First Conference on Language Modeling*, 2024.
- [27] Dacheng Li, Rulin Shao, Anze Xie, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. Lightseq: Sequence level parallelism for distributed training of long context transformers. Oct 2023.
- [28] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch Distributed: Experiences on Accelerating Data Parallel Training, 2020.
- [29] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence Parallelism: Long Sequence Training from System Perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2391–2404. Association for Computational Linguistics, 2023.
- [30] Sungbin Lim, EUN BI YOON, Taehyun Byun, Taewon Kang, Seungwoo Kim, Kyungjae Lee, and Sungjoon Choi. Score-based generative modeling through stochastic evolution equations in hilbert spaces. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 37799–37812. Curran Associates, Inc., 2023.
- [31] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [32] Hao Liu, Matej Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *CoRR*, abs/2310.01889, 2023.
- [33] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.
- [34] Xin Ma, Yaohui Wang, Xinyuan Chen, Gengyun Jia, Ziwei Liu, Yuan-Fang Li, Cunjian Chen, and Yu Qiao. Latte: Latent diffusion transformer for video generation. *Trans. Mach. Learn. Res.*, 2025, 2025.
- [35] Zizhao Mo, Huanle Xu, and Chengzhong Xu. Heet: Accelerating elastic training in heterogeneous deep learning clusters. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 499–513. ACM, April 2024.
- [36] Kepan Nan, Rui Xie, Penghao Zhou, Tiehan Fan, Zhenheng Yang, Zhijie Chen, Xiang Li, Jian Yang, and Ying Tai. Openvid-1m: A large-scale high-quality dataset for text-to-video generation. In *ICLR*. OpenReview.net, 2025.
- [37] NVIDIA. Nvidia: Context parallelism. https://docs.nvidia.com/megatron-core/developer-guide/latest/apiguide/context_parallel.html, 2024. Accessed: 2025-08-27.
- [38] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4172–4182, 2023.
- [39] Yiran Qin, Zhelun Shi, Jiwen Yu, Xijun Wang, Enshen Zhou, Lijun Li, Zhenfei Yin, Xihui Liu, Lu Sheng, Jing Shao, Lei Bai, Wanli Ouyang, and Ruimao Zhang. Worldsimbench: Towards video generation models as world simulators. *CoRR*, abs/2410.18072, 2024.
- [40] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*, pages 1–16. IEEE Press, 2020.
- [41] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. Zero-infinity. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov 2021.
- [42] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. vdn: virtualized deep neural networks for scalable, memory-efficient neural network design. *International Symposium on Microarchitecture, International Symposium on Microarchitecture*, Oct 2016.
- [43] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- [44] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism, 2019.

- [45] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, Devi Parikh, Sonal Gupta, and Yaniv Taigman. Make-a-video: Text-to-video generation without text-video data. In *ICLR*. OpenReview.net, 2023.
- [46] Step-Video Team. Step-Video-T2V Technical Report: The Practice, Challenges, and Future of Video Foundation Model. *arXiv preprint arXiv:2502.10248*, 2025.
- [47] Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Périz, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. Sailor: Automating distributed training over dynamic, heterogeneous, and geo-distributed clusters. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 204–220. ACM, October 2025.
- [48] Xin Tan, Yuetao Chen, Yimin Jiang, Xing Chen, Kun Yan, Nan Duan, Yibo Zhu, Daxin Jiang, and Hong Xu. Dsv: Exploiting dynamic sparsity to accelerate large-scale video dit training. *arXiv preprint arXiv:2502.07590*, 2025. Submitted on 11 Feb 2025 (this version), latest version 16 Mar 2025 (v3).
- [49] The Movie Gen team @ Meta. Movie gen: A cast of media foundation models. *arXiv preprint arXiv:2410.13720*, 2025. License: arXiv.org perpetual non-exclusive license. A detailed contributor list can be found in the appendix of this paper.
- [50] Taegeon Um, Byungsoo Oh, Minyoung Kang, Woo-Yeon Lee, Goeun Kim, Dongseob Kim, Youngtaek Kim, Mohd Muzzammil, and Myeongjae Jeon. Metis: Fast automatic distributed training on heterogeneous gpus. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24)*, Santa Clara, CA, USA, July 2024. USENIX Association.
- [51] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *ICLR*. OpenReview.net, 2025.
- [52] Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, et al. Wan: Open and Advanced Large-Scale Video Generative Models, 2025.
- [53] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. Superneurons: dynamic gpu memory management for training deep neural networks. *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, Feb 2018.
- [54] Qiuhe Wang, Yukai Shi, Jiarong Ou, Rui Chen, Ke Lin, Jiahao Wang, Boyuan Jiang, Haotian Yang, Mingwu Zheng, Xin Tao, Fei Yang, Pengfei Wan, and Di Zhang. Koala-36m: A large-scale video dataset improving consistency between fine-grained conditions and video content, 2024.
- [55] Yujie Wang, Shiju Wang, Shenhan Zhu, Fangcheng Fu, Xinyi Liu, Xuefeng Xiao, Huixia Li, Jiashi Li, Faming Wu, and Bin Cui. FlexSP: Accelerating Large Language Model Training via Flexible Sequence Parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, pages 421–436. ACM, 2025.
- [56] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, Da Yin, Yuxuan Zhang, Weihang Wang, Yean Cheng, Bin Xu, Xiaotao Gu, Yuxiao Dong, and Jie Tang. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer. In *International Conference on Learning Representations (ICLR)*, 2025.
- [57] Jiwen Yu, Yiran Qin, Xintao Wang, Pengfei Wan, Di Zhang, and Xihui Liu. Gamefactory: Creating new games with generative interactive videos, 2025.
- [58] Lijun Yu, José Lezama, Nitesh B Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Agrim Gupta, Xiuye Gu, Alexander G Hauptmann, et al. Language model beats diffusion-tokenizer is key to visual generation. *arXiv preprint arXiv:2310.05737*, 2023.
- [59] Peiyuan Zhang, Haofeng Huang, Yongqi Chen, Will Lin, Zhengzhong Liu, Ion Stoica, Eric P. Xing, and Hao Zhang. Faster video diffusion with trainable sparse attention. *arXiv preprint arXiv:2505.13389*, 2025. Submitted on 19 May 2025 (this version), latest version 4 Aug 2025 (v4).
- [60] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. Open-Sora: Democratizing Efficient Video Production for All. *arxiv preprint arXiv:2412.20404*, 2024.